MICRO1 FRONTIER ENGINEERING CHALLENGE 2026

Agentic Workflows Hackathon · Project report for judges

Dungeon Repair

Correctness is free. Judgment is scarce.

An agent that repairs procedurally generated lock-and-key dungeons the way their designer would have — choosing among repairs a solver has already proven correct, and explaining the choice.

42–47%
AGENT INTENT RECOVERY
seven runs: 32 to 36 of 77

33.8%
BEST BASELINE
26/77, fully deterministic

77/77
STILL COMPLETABLE
guaranteed by construction

$0.0051
PER REPAIR
27s median wall time

Author	Kabyik Kayal
Submitted	31 August 2026
Repository	github.com/Kabyik-Kayal/dungeon-repair
Model	openai/gpt-5.6-luna, via litellm
Coding agent	Claude Code (required disclosure – see §12)
Evaluation	77 cases · 4 methods · 7 agent runs · 539 trajectories

EXECUTIVE SUMMARY

What this is, and what it found

A procedurally generated dungeon can be structurally sound and still impossible to finish. The key for a door generates behind that door. A corridor gets dropped and a wing is orphaned. Three locked doors sit on the critical path and the generator placed two keys. A flood fill does not catch this: once keys are consumed and switches change world state, whether a room is reachable depends on what the player is carrying and what they have already spent. It is a search over player state, not plain connectivity.

This project began as the obvious agent: diagnose the failure, propose a fix, verify it, retry. Before building that, the null hypothesis was measured — enumerate every single edit in a small vocabulary and ask a breadth-first solver about each one. It repaired every broken level tested, deterministically, in under a second, for no API spend.

THE FINDING THAT DEFINED THE PROJECT

Making a level completable is not an agentic problem. But there is a **median of 60 provably-correct repairs per broken level** (min 6, max 612), and the deterministic method picks a bad one: it reconnects a severed corridor by joining two rooms on opposite sides of the map. The level passes the check and is still unshippable.

So the architecture inverted. **The solver is the oracle. Enumeration is the candidate generator. The agent never has to make the level completable — it chooses among repairs that are already proven correct, and explains the choice.** It cannot ship a broken level, because `submit` refuses any edit the solver has not verified.

Measured on 77 seeded corruptions of real shipped Zelda dungeons, against three baselines including a deliberately strengthened deterministic repairer:

Method	Completable	Intent recovery	Layout kept	s / case	$ / case
Rejection sampling (reroll the level)	77/77	0/77 (0.0%)	0.034	0.00	$0
Single prompt, no tools	**61/77**	23/77 (29.9%)	0.961	29.9	$0.0035
Enumerate, take first valid	77/77	26/77 (33.8%)	0.965	0.06	$0
The agent	**77/77**	**34/77 (44.2%)**	**0.966**	**27.4**	**$0.0051**

Primary metric is intent recovery: exact match against the seeded edit that broke the level. Completability is a gate, not an achievement. Full run: $0.66. Regenerate with `dungeon-repair compare`.

READ THE HEADLINE WITH ITS ERROR BAR

The agent is not deterministic — sampling parameters are unavailable on current reasoning models. Across **seven full runs** it has scored 32, 34, 34, 34, 34, 35 and 36 of 77: call it **42–47%**, never a single figure. The baseline is fully deterministic at 26/77, so the margin is 6–10 cases in every run, but the agent's own number moves by up to four — §9 and §10 are about why.

HOW TO READ THIS DOCUMENT

SECTION 1

The problem, and who has it

Intended user. Solo and small-team developers who generate levels procedurally and ship them without a human walking every seed — dungeon crawlers, roguelikes, metroidvanias, anything where progress is gated by keys, switches, and locked doors. A studio with a QA department brute-forces this with people. A two-person team cannot.

The bottleneck. Verifying that a generated level is completable is not connectivity checking. Small keys are fungible and consumed, so spending one on the wrong door can strand the player permanently; a boss key or a key item flips world state. Reachability is therefore a function of the player's inventory and spend history, and a flood fill over the room graph will happily certify a dungeon that cannot be finished.

What teams do today

Today's practice	Why it fails at scale
Playtest every seed by hand	Linear in seeds. The thing procedural generation exists to avoid.
Reject and reroll until a level passes	Throws away the hand-tuned layout, and teaches you nothing about what was wrong. Measured here at **0.034 layout similarity** – it is a different dungeon.
Constrain the generator so only solvable levels come out (ASP / `clingo`)	Genuinely strong, and the most credible alternative. But it cannot repair a level that already exists and has been tuned by a person.

Audited in Stage 1 of the changelog against existing products. Chess blunder coaching, crash triage, visual regression, and mod load-order conflicts all have mature tools and were dropped for that reason; playability repair for arbitrary lock-and-key mechanics does not.

Why this is worth an agent at all

It is not, for the part everyone assumes. That is the finding in §2, and it is the reason this project has a shape worth defending: the agentic surface is smaller and better-defined than the original plan assumed, and the surrounding harness makes the agent incapable of producing an invalid answer. What is left for the model is the only part a solver cannot do — *which* of sixty correct repairs is the one the designer actually intended.

The data

The Video Game Level Corpus (MIT) — room graphs transcribed from three shipped Zelda games. All three are used, not one: boss keys and numbered switches appear *only* in *Link to the Past* and *Link's Awakening*, so building on base Zelda alone leaves that logic untested. The corpus is cloned by a script at a pinned commit, not vendored.

38
DUNGEON GRAPHS
LoZ 18 · LttP 12 · LA 8

31
VERIFY AS COMPLETABLE
the eval set is built from these

7
FAIL – AND SHOULD
transcription errors in the corpus

101
SOFT-LOCKED DOORS
in both directions – see §4

The 7 failures are not solver bugs. `LttP_3` encodes one small key against three key-locked doors, all on the critical path — no ordering of spends finishes it, and the shipped game plainly does. Surfacing real errors in a published research dataset is a side effect of having a solver worth trusting.

SECTION 2

The experiment that deleted the original design

Changelog Stage 5 — the most important twenty minutes in the project

The plan was a conventional repair agent: read the diagnosis, propose a fix, call the solver, retry on rejection. Before writing it, one question was worth answering cheaply — *how hard is this actually?* So the null hypothesis got measured first: enumerate every single edit in a four-verb vocabulary (`move_key`, `add_key`, `unlock`, `add_door`) and ask the solver about each.

```
# eval/results/00_baseline_probe.txt
repaired      : every seeded corruption tested
searched      : ~100 candidates per level
time          : ~1.6 s per level
cost          : $0.00
valid repairs : median 60 per broken level   (min 6, max 612)
```

An LLM asked to make a level completable is competing with something faster, cheaper, and deterministic, and losing on all three axes at once. The original design was deleted before it was written.

But the same run produced the number the whole project now rests on. **Sixty correct answers per level.** "Does a repair exist" is trivial; "which repair" is untouched. And the deterministic method picks badly in a specific, diagnosable way — ranked least-invasive-first it will reconnect a severed corridor by joining two rooms six doors apart. That verifies. It is also a teleport, and no designer would ship it.

> *"The solver is the oracle. Enumeration is the candidate generator. The agent never has to make the level completable — it chooses among repairs that are already proven correct, and explains the choice."*

Three consequences worth naming

Correctness became a property of the harness, not a hope about the model. `submit` checks the proposed edit against the verified set and refuses anything not on it. The refusal is returned to the model as a tool result, so the loop is a verification loop rather than a suggestion box. A test drives a stub model that only ever proposes invalid repairs and asserts the run ends with *no answer* rather than a bad one.

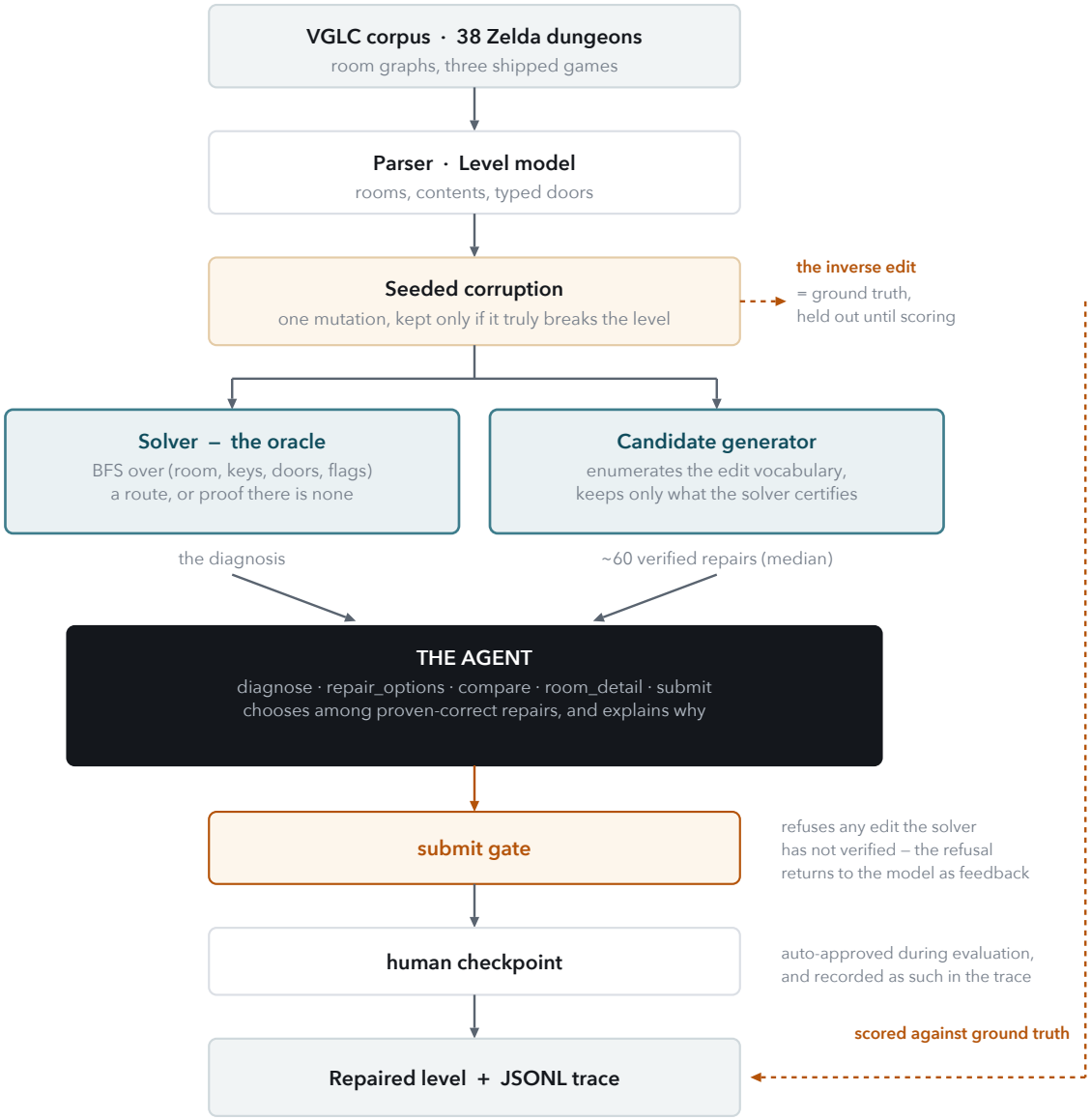
The baseline got stronger, on purpose. The first measurement of the deterministic repairer used an arbitrary enumeration order and scored **0%** intent recovery. That number was tempting and wrong to publish — the order was doing the damage. Re-ranked least-invasive-first, which is what an engineer would actually write, it scores **33.8%**. A baseline chosen to lose is not evidence.

The metric became objective rather than a taste test. Because every corruption is a seeded, known mutation, the designer's intended repair is its exact inverse. "Which fix is best" stops being an opinion and becomes a string comparison. Nothing else in this project would be checkable without that.

> **WHAT A JUDGE SHOULD TAKE FROM THIS SECTION**
>
> The strongest engineering decision here was to *remove* the agent from the part of the problem it was originally designed for, and it was made on evidence collected before any of that code existed. The measurement cost twenty minutes and zero dollars, and it is the reason the rest of the system has a defensible shape instead of an LLM wrapped around a solved problem.

SECTION 3
Architecture

Everything flows one direction, and the load-bearing property is that nothing downstream of the solver can produce a level the solver has not approved.



Solid arrows carry data. The dashed path is the ground truth: the exact inverse of the seeded mutation, withheld from every method and used only at scoring time.

The code

3,132 lines of Python in `src/`, 520 in `tests/`. Two runtime dependencies: `litellm` for model access and `pytest` for tests. Nothing else.

Module	Responsibility	The one thing worth knowing
`vglc.py`	Parse Graphviz `.dot` room graphs	The obvious node regex also matches edge *targets*, silently overwriting room contents. Edge statements are blanked first.
`level.py`	The `Level` model – rooms, contents, typed doors	Doors are directional and typed; the same pair can be open one way and locked the other, and the corpus uses that.
`solver.py`	BFS over player state; route or proof of impossibility	State is `(room, key bitmask, door bitmask, flags)`. Sets made a 62-room dungeon hang.
`corrupt.py`	Seeded mutations, retained only if they truly break the level	Round-robin over kinds with a per-kind retry budget – uniform sampling produced a badly lopsided set.
`candidates.py`	Enumerate the edit vocabulary, keep what the solver certifies	This is the entire correctness guarantee, in 120 lines.
`edits.py`	The four-verb edit vocabulary and its canonical form	Canonicalising door edits had a swap bug that mangled every descending room pair.
`agent/tools.py`	Five tools, their schemas, and the submit gate	The largest module (519 lines) – because the tools, not the prompt, are where the design work is.
`agent/loop.py`	Tool-calling loop, budget, trace emission	Every turn is written to JSONL as it happens, including the human checkpoint.
`metrics.py`	Intent recovery, completability, layout similarity, spend	Layout similarity is Jaccard over locked doors and room contents.
`llm.py`	litellm wrapper, cost accounting, timeouts	Prices come from litellm's own map, versioned with the pinned library, rather than a hand-copied table that can only drift.

SECTION 4

The solver

Everything else in this system is only as trustworthy as this

Why the simple approach does not work. Flood fill answers "is room X connected to the start," which is the wrong question. Small keys are fungible and consumed: three keys and three locked doors is not automatically fine, because spending the first key on the wrong door can strand the player forever. Reachability depends on inventory and spend history.

What it tracks. A search node is `(room, collected-key bitmask, opened-door bitmask, flags)`. The first attempt used Python sets for the collected rooms; it was correct and made a 62-room dungeon hang, because the state space is exponential and a set is not hashable-cheap. Bitmasks fixed it. All 38 dungeons now resolve in well under a second; the whole corpus verifies in 0.5 s.

Two rules that had to be discovered by experiment

Question	First answer	Evidence	Rule
What does the legend's "soft-locked" door mean?	Impassable	Only **6 of 38** shipped dungeons came out completable. 101 doors carry the flag in *both* directions – which cannot mean impassable, that would wall a room off from itself.	**Passable.** One flag, 25 dungeons.
Is a key consumed, or is it a permission?	Permission (holding one opens any lock)	The solver certified levels that cannot be finished, because it never modelled spending the key on the wrong door first.	**Consumed.** Opened doors carry their own bitmask.

Both are asserted directly by `tests/test_solver.py::test_shipped_corpus_regression`, which pins the 31/38 figure. If a future change to the solver moves that number, the test fails and every result in this report is suspect – which is the point of pinning it.

What it hands back

Not a boolean. The diagnosis is what the agent reads first, and it is written to be diagnostic rather than merely negative — which rooms are reachable, which keys can never be collected, which doors were reached but never passable, and whether the key economy balances.

```
Winnable: no
Why: goal room 11 is not reachable; small key(s) in room(s) 17 cannot be
     collected; out of keys at door(s) 1-13, 10-14, 12-16, 13-1, 14-10, 16-18, 8-4
Reachable rooms (16): 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 13, 14, 16, 18
Unreachable rooms (3): 11, 15, 17
Doors reached but never passable:
  1 -> 13 (no small key in hand)
  10 -> 14 (no small key in hand)
Key economy: 6 small key(s) for 6 key-locked door(s)
```

Real output from `eval/traces/agent__LoZ_1_1.jsonl`. Note the last line: six keys for six locks means this is not a key shortage, which is exactly the inference the agent draws from it in Appendix A.

SECTION 5
The agent

The agent receives a description of the broken dungeon and five tools. It is specifically **not** given the job of finding a fix — the tool that lists fixes only ever returns ones the solver has already approved. Its entire budget goes on the question the solver cannot answer.

Tool	Answers	Design note
`diagnose`	Why can this not be finished?	Leads with stranded keys, ahead of the blocked-door list. This ordering was itself a tested hypothesis (§9).
`repair_options`	What repairs are available?	Filterable by kind, by room, by hop distance – because the largest candidate set in the evaluation is **612** repairs and dumping it answers one question and no others.
`compare`	How do these specific repairs differ?	The tool that carries the design judgment: hop distance between the rooms involved, whether topology changes, the key-to-lock economy afterwards, the new winning route, and how many rooms open up. **It deliberately accepts unverified edits and says plainly that they do not work**, so the agent can test its own idea.
`room_detail`	What is in this room, what connects to it, how far is it?	Hop distance from start, because the graphs carry no coordinates (§12).
`submit`	Here is my answer and my reasoning	**The gate.** Checks against the verified set; a refusal returns to the model as a tool result and it tries again.
`design_rhythm`	What do these designers' other dungeons do with keys?	Added in Stage 16 (§10), opt-in. Motifs mined with this dungeon held out, each printed with its measured lift because they are all weak. Offered only where a key edit is legal.

What the instructions spend words on

Not correctness — that is already handled by the harness, and a prompt arguing for it wastes the model's attention. The prompt instead defines what makes a repair *good*, and asks for one kind of reasoning: work backwards from symptom to cause. *What single generation mistake would produce exactly this failure — not merely some failure?*

> VERBATIM, FROM THE SYSTEM PROMPT
>
> **Locality.** Real corridors join rooms that sit next to each other. A new passage between rooms six doors apart is a teleport, not a repair, even though it verifies.
>
> **Restore, do not invent.** Prefer undoing whatever the generator got wrong over adding structure the designer never drew.
>
> `unlock` **is blunt.** It clears *every* requirement on a door, not just a small-key lock.

Human in the loop

Nothing is applied without approval. Interactively the agent asks; during automated evaluation it auto-approves **and the trace records which of the two happened**, as a `human_checkpoint` event carrying `"automatic": true` — so the distinction is auditable after the fact rather than asserted here.

Traces

Every run writes JSONL as it happens: the instructions, each model turn with its token usage, every tool call and result, the checkpoint, and the scored outcome. **539 trajectories across seven full runs** are submitted in `eval/traces/` — the full 77 for the shipped configuration, the diagnosis-gated arm, and five runs of the Stage 16 arm. Appendix A walks one end to end.

SECTION 6

Manufacturing ground truth

The part that makes any of this measurable

"Which repair is better" is a taste question, and taste questions do not produce numbers a judge can check. The trick is to **break the level yourself**. Take a dungeon that verifies as completable, apply one seeded mutation, and keep it only if the solver confirms it genuinely broke — so every case sits on the critical path. The designer's intended repair is then the exact inverse of a mutation you already know.

Corruption	What breaks	Ground-truth repair	Cases
`displaced_key`	A small key generates in the wrong room	Move it back	15
`severed_corridor`	A passage is dropped	Reopen it	31
`spurious_lock`	A door that should be open is key-locked	Unlock it	31

The set was quietly unbalanced, and that was a bug

Sampling corruption kinds uniformly gave 55 severed corridors, 21 spurious locks and **7** displaced keys. Cutting a corridor breaks a dungeon far more often than moving a key does, so uniform sampling of *attempts* produces a lopsided set of *cases* — and it buried the displaced key, which is the failure the problem statement actually opens with. Fixed by round-robin over kinds with a per-kind retry budget.

The deliberately hard case

`spiral_keep#hard` is authored, not sampled. The keep repeats one rhythm three times — an alcove holding the key for the gate directly ahead — and the third key generates behind its own gate. **115 repairs verify. Ten of them move the stranded key. One puts it back in its alcove.** It is designed so that solving it requires reading the level's design rhythm, which is precisely the signal none of the tools expose.

Metrics

Intent recovery (primary)	Did the chosen repair exactly match the edit that broke the level. **Exact match, on purpose.** A repair that is arguably as good but different scores zero. This is strict, and it is the only version of the metric that cannot be argued with.
Completability	A gate, not an achievement. Every method with solver access clears it by construction. It is reported anyway, because one method does not have solver access and that is the whole point of reporting it.
Layout preservation	Jaccard similarity of doors-with-locks and room contents against the original — the number that exposes rejection sampling as a different dungeon rather than a repair.
Wall time and cost per case	Measured from actual per-call token counts, not a list-price estimate.

Tests

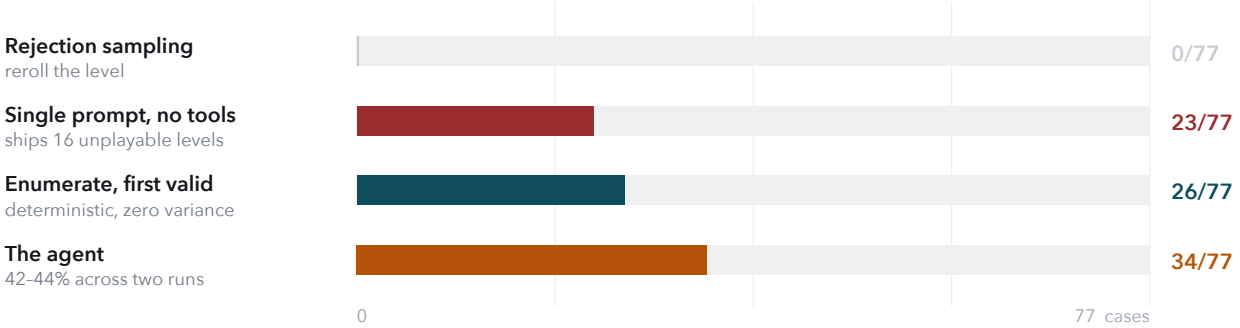
30 tests, none of which require an API key. The model-facing loop is tested against a stub returning scripted tool calls, so the agent's control flow — including the case where every proposal is invalid and the run must end with *no* answer — is verified deterministically and for free.

SECTION 7

Results

77 cases · four methods · one primary metric

Method	Completable	Intent recovery	Layout	s / case	$ / case	Total $
Rejection sampling	77/77	0/77 (0.0%)	0.034	0.00	$0	$0
Single prompt, no tools	**61/77**	23/77 (29.9%)	0.961	29.9	$0.0035	$0.27
Enumerate, first valid	77/77	26/77 (33.8%)	0.965	0.06	$0	$0
The agent	**77/77**	**34/77 (44.2%)**	**0.966**	**27.4**	**$0.0051**	**$0.40**



Intent recovery — exact match against the seeded edit. Higher is better.

THE ROW TO LOOK AT TWICE IS NOT THE AGENT'S

The single-prompt baseline has no solver, so it is the only method here that can ship a broken level — and it does: **16 of its 77 repairs leave the dungeon impossible to finish.** Its 29.9% sits within a few points of the deterministic repairer's 33.8%, which makes it look competitive. It is not. A fifth of its answers are unusable.

This is the clearest evidence in the project that intent recovery must be read next to the completability gate and never alone — and the reason the gate is reported in a column of its own despite three of four methods passing it by construction.

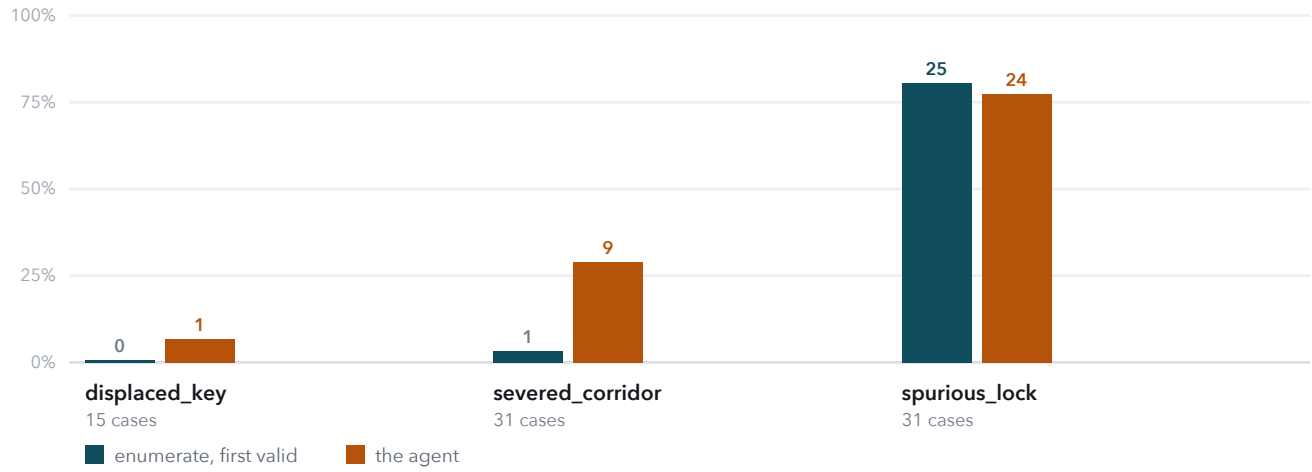
Cost and time

The agent costs **half a cent and 27 seconds** per repair. A full four-method comparison over 77 cases is **$0.66** and about 75 minutes with 4 workers. Total model spend across the entire project — every experiment, every arm, every re-run — was approximately **$2.30**. The two deterministic methods are free; only the two model-backed ones spend anything, which is why `dungeon-repair check-model` exists: one cheap call that confirms credentials, the model string, and tool-calling support before a run fails identically 77 times.

SECTION 8

Where the win comes from — and where it does not

The aggregate number is 44.2% against 33.8%, and it hides almost everything interesting. Split by corruption kind, the agent's advantage turns out to live in one column.



Intent recovery as a share of cases in each kind. Bar labels are absolute counts.

Corruption kind	n	Rejection	Single prompt	First valid	Agent
`displaced_key`	15	0	1	0	**1**
`severed_corridor`	31	0	5	1	**9**
`spurious_lock`	31	0	17	**25**	24

Read honestly, that breakdown is the whole argument

Spurious locks: the agent is level with a script. 24 against 25. Least-invasive-first ordering means the deterministic repairer tries `unlock` first, and on a spuriously locked door that is simply the right answer. There is no agentic value here and the numbers say so.

Severed corridors: this is the entire win. 1 to 9. “How far apart are these two rooms” is a genuinely useful signal, the `compare` tool exposes it, and the agent uses it to reject the teleport the script happily ships.

Displaced keys: 1 of 15, and this is the honest headline. The failure is sharper than the number. On 13 of those 15 the agent does not pick the *wrong room* to put the key back in — **it does not choose to move the key at all.** It unlocks a door instead. That rebalances keys against locks, it passes the gate, and it is wrong: the dungeon now has a lock the designer drew and the player never has to solve.

> THIS IS THE FAILURE THE NEXT SECTION CHASES
>
> Three experiments were run against it. All three failed to move the score, and the third one explained why the first two could never have been believed in the first place.

SECTION 9

Three experiments and a control

Changelog Stages 13–15 – including the one that invalidated a published claim

Experiment 1 — rebalance the tools (Stage 13). The first full run showed the agent choosing `unlock` on 48 of 77 cases when only 31 call for one. Three biases in our own code all pointed the same way: `repair_options` returned candidates in enumeration order — which is unlock-first, because that ordering exists to strengthen the baseline — *and truncated to the first 25*, so on many cases the agent never saw the alternatives; the prompt's "restore, do not invent" rule used unlocking as its worked example; and nothing told the model that `unlock` clears *every* requirement on a door. Nine of its 48 unlocks had demolished deliberate structure — seven impassable walls, a boss-key door, a key-item gate.

The fix was balanced round-robin sampling across kinds, a neutral wording of the rule, and an explicit warning on any unlock that removes more than a key lock. Unlock choices fell from 48 to 40, exactly as intended. **The score moved by exactly nothing: 34/77 either way.**

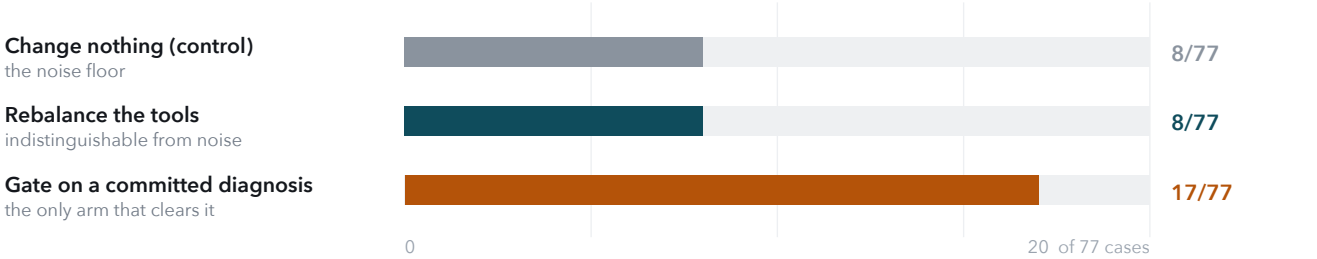
Experiment 2 — force a diagnosis first (Stage 14). If the agent's answer tracks tool prominence rather than evidence, perhaps it never had to form a view: `repair_options` was available from turn one, so it could browse first and reason afterwards. So the option tools were gated behind a commitment — the agent must call `hypothesise(repair_kind, rooms, reasoning)`, predicting the bug in the vocabulary it already had, before anything else will answer. Revision is allowed; the *first* hypothesis is what gets scored. Gating made the score *worse* — 31/77 — though exact McNemar on the 17 discordant pairs gives $p = 0.63$, so the drop is not distinguishable from chance.

Arm	Intent recovery	Chose `unlock`	Chose `move_key`	$ / run
v1 – unlock-first (as first measured)	34/77	48	2	$0.38
v2 – rebalanced tools *(shipped)*	34/77	40	8	$0.40
v2 repeat – nothing changed (control)	**32/77**	**41**	**7**	**$0.39**
v3 – diagnosis gate	31/77	36	10	$0.45

Experiment 3 – the one that should have been first

Every comparison above is a single run against a single run. Sampling parameters are unavailable on current reasoning models, so run-to-run variation cannot be tuned away — only measured. So the shipped configuration was re-run **unchanged** and compared against itself.

Pair	Intervention	Intent	Discordant pairs	Same repair chosen	p
v2 vs v2-repeat	**none (control)**	**34 → 32**	**8**	**49/77 (64%)**	**0.73**
v1 vs v2	rebalanced tools	34 → 34	8	46/77	1.00
v2 vs v3	diagnosis gate	34 → 31	17	36/77	0.63



Cases where the two runs chose a different repair. The control run is the bar every other bar has to beat, and it was measured last instead of first.

WHAT THIS INVALIDATED, AND WHAT SURVIVED

Changing nothing moves two cases and flips eight. The agent chooses the identical repair on only **64%** of cases given identical input — 52% on severed corridors. Experiment 1 also flipped eight. Everything the original write-up inferred from *which* eight — that the agent had redistributed its answers, that the losses clustered revealingly — was reading tea leaves in its own variance. That claim is corrected in place in Stage 13 rather than deleted.

What reproduces is the behavioural shift, not the score. The repeat run chose 41 unlocks against v2's 40 and nowhere near v1's 48. Rebalancing the tools reliably changed what the agent reaches for and reliably failed to change how often it is right.

The comparison against the baseline survives easily. `first_valid` is deterministic at 26/77 with zero variance; the agent scored 34 and 32. The margin is 6–8 cases in every run and never in question. What is in question is any claim of a difference *smaller* than that.

SECTION 10

Giving the agent what it lacked

Changelog Stage 16 – five more runs, two falsified predictions, one result

§9 left the failure located but unfixed: the agent names the right kind of bug 76.6% of the time and produces the right repair 40.3% of the time. So two components were built for exactly that gap — and, first, the ceiling was measured.

Where the headroom is not

Three independent attempts to beat the agent deterministically on the corridor cases, all at $0. None of them did:

Method, on `severed_corridor` (n = 31)	Intent recovery
Enumerate, first valid	1
Learned link prediction (leave-one-level-out logistic)	2
Single prompt, no tools	5
Hand-built structural ranker (boundary, id gap, degree)	6
The agent	**9**

A perfect kind classifier plus the best of those rankers scores 33/77 – the same as the agent. **27 of the 31 severed corridors have their endpoints in different connected components**, so every path-based link-prediction feature is identically zero.

What the design memory actually measured

The changelog's own explanation of the failure was that keys sit in an alcove before the gate they open. Measured, that is **false**: 102 of 127 keys sit beyond their nearest locked door. What is true is duller — 76% of key rooms hold an enemy against a 53% base rate (lift 1.45), 28% are dead ends against 20%, and key items and small keys almost never share a room (lift 0.24). And the motifs **narrow without ranking**: as a log-prior they recover 2 of 15 displaced keys, no better than plain topology, while "the home room holds an enemy" keeps 13 of 24 candidates and retains the right one 13 times in 15. So the memory ships as evidence with its lift printed beside it, never as an ordering — and every statistic is mined with the dungeon under repair held out, which a test pins.

Five runs, and the one column that reproduces

Run	Total	The 23 sole-unlock cases	Everything else (54)
baseline	34	18	16
baseline, unchanged re-run	32	19	13
+ memory, both notes	35	**21**	14
+ memory, both notes (repeat)	34	**21**	13
+ memory gated to key cases	34	**21**	13
one note removed	36	**21**	15
one note removed (repeat)	34	**21**	13

The agent decides every case; nothing bypasses it. `design_rhythm` was called in 77 of 77 trajectories.

ONE FINDING KEPT, EVERYTHING ELSE REPORTED AS NULL

21 of 23 is the deterministic ceiling of that rule — the other two are displaced keys it mislabels. Handed one fact that closes the question, the agent reaches that ceiling five times out of five, against 18 and 19 without it. That is the only intervention in this project that produced a reproducible gain.

Nothing else moved. The headline means are 34.6 against 33, inside a noise floor already measured at eight flipped cases; the final configuration against itself is 8 discordant and 61% identical. The design memory did not shift the displaced key it was built for: 2, 1, 1, 3, 1 against 1 and 0.

Two predictions were registered in advance and both were falsified. That the memory was distracting the agent on corridor cases — gating it did not restore them. That a second note was the culprit — removing it gave 36 once and 34 on repeat. Both are kept in the changelog.

What the seven runs say, which is worth more than the score

The agent converts facts and ignores hints. A note that determines the answer is worth +3, reproducibly, up to the exact limit of what the fact supports. A design motif measured at lift 1.45 is worth nothing. A true but non-determining note — “this must be a dropped corridor” — is worth nothing either, and may cost. Stage 13 found that tool presentation moves behaviour without moving accuracy; this narrows it to what does: information that closes the question.

That redirects the whole improvement programme. The way to improve this agent is not better hints, richer context or more persuasive instructions — it is auditing the pipeline for other places where the surrounding machinery can settle a question outright, and handing those over as facts.

SECTION 11
The failure mode, and the lesson

The agent knows what broke and cannot work out how to put it back.

The diagnosis gate scored worse, but it bought a second, independent measurement that no other arm could produce: what the agent believed *before it saw a single option*, scored separately from what it finally did.

76.6%
DIAGNOSIS CORRECT
59/77 – named the right kind of bug

40.3%
REPAIR CORRECT
31/77 – produced the exact edit

0 of 18
WRONG DIAGNOSIS RECOVERED
diagnosis is a hard gate, not a prior

The agent diagnoses roughly twice as well as it repairs. The 28 cases in between — diagnosed correctly, repaired wrongly — are the actual failure, and they are not a diagnosis problem at all. Two earlier explanations were tested and rejected, which is why this one is worth trusting more: the claim that the diagnosis buries the evidence is false (`_diagnose` already leads with the stranded-key line), and the claim that the option list biases the choice is true but not decisive (rebalancing moved unlock choices 48 → 40 and the score not at all).

> THE HARD CASE, IN THE AGENT'S OWN WORDS
>
> "*The failure is a circular key dependency: rooms 10 and 11 provide the first two keys, but the third small key and boss key are in room 13, beyond the gate.*"
>
> That is a correct and precise diagnosis, written before the agent saw a single repair option. It then proposed **removing the gate** rather than returning the key. It diagnosed a stranded key and prescribed demolition.

Why – and it is not a prompting problem

Identifying a bug and identifying its *inverse* are different skills, and this system only supports the first. Choosing the room a key came from requires reading the dungeon's design rhythm — that this keep puts an alcove before every gate, so a stranded key *belongs* in the alcove before the gate it opens. Nothing in the tools exposes rhythm. They expose distance, topology, and key counts. The failure is in the tool surface, not the wording.

HOT TAKE

Measure your noise floor first

The lesson worth carrying out of this project into the next one

Measure your noise floor before you believe a single thing your A/B told you.

This project ran three interventions on an agent and wrote up two of them before measuring how much the agent disagrees with *itself*. The answer, when finally measured, was: on identical input it picks the same repair 64% of the time, and an unchanged re-run moves the headline by two cases and flips eight.

Eight. The first intervention also flipped eight. The control cost $0.39 and one re-run, and it should have been the *first* experiment, not the fifth. A/B testing an agent without a same-config baseline is not evidence — and the failure is seductive precisely because the churn is real, the cases are individually inspectable, and each flipped case has a story you can tell about it.

Three things survived the control, and are worth more for having survived it:

— **The architecture works.** Solver as oracle, enumeration as candidate generator, agent spending its whole budget on judgment: 42–44% intent recovery against a deterministic 33.8%, a 6–8 case margin in every run, and a completability gate it cannot fail by construction.

— **Tool presentation reliably changes behaviour and reliably does not change accuracy.** Unlock choices fell 48 → 40 (41 on repeat) → 36 across the arms, a stable reproducible shift. The score never left 31–34. You can steer what an agent reaches for far more easily than you can make it right.

— **The real gap is not diagnosis.** Instrumented, the agent names the right kind of bug 76.6% of the time and produces the right repair 40.3% of the time, and 18 of 18 wrong diagnoses are fatal. Naming a fault and inverting it are different skills.

None of which would be checkable at all without manufactured ground truth. Seeding the corruption is what turned “which fix is best” from a taste test into a number — and, once there was a number, what made it possible to discover that the number was noisier than the effects being claimed on top of it.

SECTION 12

Improvement changelog

Sixteen stages, written as the work happened. Failed experiments included.

Full text, with the evidence command for every row, in `docs/CHANGELOG.md`. Stages 13–16 are failed or null experiments and are kept as such; Stage 13's original conclusion is corrected in place rather than deleted, because the correction is the most useful thing in it.

#	What was tried, and why	Evidence	Decision
1	Audit candidate problems against existing products before writing code	Chess coaching, crash triage, visual regression, mod conflicts all have mature free tools that do detection *and* the fix	**Kept** playability repair
2	The obvious DOT node regex	Also matches edge *targets*; room contents silently overwritten, nothing crashed	**Revised** – blank edges first
3	Treat "soft-locked" doors as impassable	Only 6/38 dungeons completable; 101 doors carry the flag both ways	**Revised** – passable → 31/38
4	Solver state as (room, keys held, switches)	Under-specified – certified levels that cannot be finished. Fixing it with sets hung a 62-room dungeon	**Revised** – bitmasks
5	**Measure the null hypothesis before building the agent**	Enumeration repairs everything, ~1.6 s, $0 – *and* leaves a median of 60 valid repairs to choose between	**Removed** the original design
6	First baseline used an arbitrary enumeration order	Scored 0%. The order was doing the damage, not the method	**Revised** – least-invasive-first, 33.8%
7	Sample corruption kinds uniformly	55 / 21 / 7 split – buried the displaced key, the headline failure	**Revised** – round-robin, 31/31/15
8	Canonicalise door edits; invert a displaced key with one move	A swap bug overwrote `a` before reading it, mangling every descending room pair. Baseline moved 28.9% → 33.8%	**Kept** both fixes
9	Dump all candidates into the prompt, or expose them behind tools?	Largest candidate set is 612 repairs; a dumped list answers one question and no others	**Kept** tools + submit gate
10	Hand-maintained price table in `llm.py`	litellm's own map already knew every model, and a rate probe misread a tiered price ($0.40 vs $0.20)	**Revised** – prices from litellm
11	Full evaluation, four methods, 77 cases	Agent 34/77 vs 26/77; single-prompt ships 16 broken levels; whole run $0.66	**Kept**
12	Misread a finished run as a hang and killed it	Both readings wrong – relative timestamps, and 0% CPU is network I/O. But `completion` had no timeout	**Kept** a fix for a bug that never fired
13	Rebalance the tools to kill the unlock bias	Unlock choices 48 → 40. Score 34 → **34**. Zero measured improvement	**Kept**, logged as a failed experiment
14	Gate the option tools behind a committed hypothesis	31/77 (worse, p = 0.63) – but decomposed the failure: diagnosis 76.6%, repair 40.3%, 0 of 18 wrong diagnoses recoverable	**Kept** as opt-in `--diagnose-first`
15	**Re-run the shipped config unchanged (the control)**	32/77, eight cases flipped, 64% agreement with itself. Invalidates Stage 13's case-level inference	**Kept**; earlier claims corrected in place
16	**A design memory, and the facts the candidate set already implies**	Three rankers all at or below the agent on corridors. The alcove story is false (102 of 127 keys sit past their gate). Five runs: the sole-unlock subset goes 18, 19 21 every time; the total does not move	**Kept** the note that reproduces; memory opt-in; two predictions falsified and recorded

SECTION 13

Reproduction

Clean machine to full comparison table

Python 3.11+, git, ~120 MB of disk (almost all of it the corpus). An API key is needed only for the two model-backed methods. Everything else — the solver, the corpus verification, the case set, the two deterministic baselines, and all 44 tests — runs for free. Run the model-backed arms **one at a time**: two concurrent runs at four workers is enough to trigger rate limiting.

`docs/REPRODUCE.md` has the version-pinned walkthrough with expected stdout for every command, and `docs/GUIDE.md` is the module-by-module explainer with the reading list.

```
$ git clone <repo> dungeon-repair && cd dungeon-repair
$ python3 -m venv .venv && . .venv/bin/activate && pip install -e ".[dev]"
$ bash scripts/fetch_data.sh          # VGLC at pinned commit 0e86a8f3

$ pytest -q                           # 44 tests, no API key          ~5 s   $0
$ dungeon-repair verify               # 31/38 shipped dungeons        ~0.5 s $0
$ dungeon-repair build-cases          # 76 seeded cases, deterministic ~6 s   $0
$ python scripts/make_hard_case.py    # + spiral_keep#hard            ~1 s   $0

$ dungeon-repair run rejection        # baseline 1                    ~1 s   $0
$ dungeon-repair run first_valid      # baseline 2 (the one to beat)  ~5 s   $0

# from here on, model access is required (.env: OPENAI_API_KEY=...)
$ dungeon-repair check-model          # credentials + tool calling    ~2 s   <$0.01
$ dungeon-repair run single_prompt --workers 4                                ~10 m  ~$0.27
$ dungeon-repair run agent --workers 4                                        ~9 m   ~$0.40
$ dungeon-repair run agent --workers 4 --route --memory   # the Stage 16 arm  ~$0.40
$ dungeon-repair compare              # regenerates the results table  instant $0
```

Step	Runtime	Cost	Deterministic?
Tests, verify, build cases, both free baselines	< 20 s total	$0	**Yes** – same seed, same 77 cases, asserted by a test
`check-model`	~2 s	< $0.01	n/a
`run single_prompt`	~10 min (4 workers)	~$0.27	No – expect 21–25 intent hits, 60–63 completable
`run agent`	~9 min (4 workers)	~$0.40	No – **expect 32–36/77**, 77/77 completable
`run agent --route --memory`	~9 min	~$0.40	No – same range; the sole-unlock subset is 21/23 every run
Everything, end to end	~20 min	**~$0.67**	Partially – see §9

WHAT A REPRODUCTION SHOULD AND SHOULD NOT MATCH

The deterministic half must match **exactly**: 31/38 dungeons verify, 77 cases are built, `first_valid` scores 26/77. If any of those differ, something is wrong and the tests will say which.

The agent's own figure will not match exactly, and this report would be dishonest if it claimed otherwise — §9 measures that noise floor at eight flipped cases. **Expect 32–36 of 77, and 77/77 completable every time.** The completability gate is structural, not statistical.

One dependency note that is a real defect: **litellm imports `tenacity` on its retry path without declaring it**, so a clean install works until the first request that needs retrying. Now pinned in `pyproject.toml`.

SECTION 14

Limitations, disclosure, and what comes next

Limitations

No coordinates – a ceiling, not an inconvenience	VGLC graphs record connectivity, not geometry. **27 of the 31 severed corridors have their endpoints in different connected components**, so every path-based link-prediction feature is identically zero. Recovering the geometry from VGLC's tile maps was tried and abandoned — a different transcription, disagreeing on room count in 17 of 18 dungeons. On purpose-built coordinate-carrying dungeons, requiring a corridor to join adjacent rooms cuts 73 candidates to 4.4 with perfect recall, and a uniform pick among those four is still 25%. Geometry triples the odds; it does not settle the question.
One edit per case	Corruptions are single mutations and repairs are single edits. Compound breakage is out of scope, and the candidate enumeration does not currently search pairs.
Zelda-family mechanics	Small keys, boss keys, key items, switches. A new mechanic needs a solver rule today; there is no generic constraint language yet.
Exact-match scoring	A repair that is arguably as good as the original but different scores zero. Strict on purpose — it is the only version of the metric that cannot be argued with, and it understates every method equally.
Single model, two runs	All figures are `openai/gpt-5.6-luna`. Cross-model variance is unmeasured, and §9 shows that even same-model variance is larger than most of the effects that were chased.

What comes next, in priority order

— **Find more facts, not more hints.** This used to read “expose design rhythm to the agent”. §10 records what happened when we did: the motifs are real, weak, and the agent ignores them. What worked was telling it something the verified set already determined. The productive direction is auditing the pipeline for other questions the surrounding machinery can settle outright — and there may not be many, which is itself the finding.

— **Run every arm three times.** §9 establishes that single-run A/B on 77 cases cannot resolve a difference below roughly eight cases. Nothing further should be concluded from one run of anything.

— **Compound corruptions.** Two simultaneous mutations, and a candidate generator that searches edit pairs — which is where the enumeration approach starts to cost real time and the agent's ability to prune becomes worth something.

Disclosure

Built during the hackathon	Everything in `src/`, `tests/`, `scripts/`, `eval/` and all documentation.
Pre-existing	The VGLC corpus (MIT, cloned by script at a pinned commit, not redistributed), and the libraries pinned in `pyproject.toml`.
Coding agent used	Claude Code, throughout. Its use is recorded in the changelog; the repair agent's own 539 agent trajectories across seven runs are submitted under `eval/traces/`, and every arm's results under `eval/results/archive/`, indexed.
Data ethics	A public MIT-licensed research corpus of room graphs. No personal data, no credentials, nothing scraped.
Consequential actions	None. The system edits an in-memory level graph and writes files under `eval/`. Every repair passes a human checkpoint; auto-approval during evaluation is recorded as such.
Licence	Code MIT. The corpus is MIT and belongs to its authors.

APPENDIX A

One agent trajectory, end to end

eval/traces/agent__LoZ_1_1.jsonl – a severed corridor, diagnosed correctly and repaired wrongly

Chosen deliberately: not a success case. It shows the loop working exactly as designed and still producing a miss.

1 · INSTRUCTIONS t = 0.005 s

The broken dungeon as text — 19 rooms, 19 doors, every room's contents and every door's type — plus the system prompt from §5.

```
"Diagnose what single generation mistake produced this failure, then choose the repair that undoes it."
```

2 · TOOL CALLS t = 3.5 s · 1,712 in / 122 out

Two calls in parallel: `diagnose()` and `room_detail()` on all 19 rooms.

Result: goal room 11 unreachable; rooms 11, 15, 17 sealed off; *key economy: 6 small keys for 6 key-locked doors*.

3 · HYPOTHESIS (scored separately) t = 14.7 s · 674 output tokens

```
hypothesise(repair_kind="add_door", rooms=["2","11"])
```

"The failure is a structural disconnection: rooms 11, 15 and 17 form a sealed boss/goal wing, while all six small keys already exist and the reachable side has exactly six key-locked doors. That points to one corridor being omitted, not a key shortage or a bad lock."

This is the right kind of bug, reasoned to from the key economy, before seeing a single option. The gate now unlocks the option tools.

4 · OPTIONS t = 18.8 s

Four filtered queries. Two return nothing — `unlock` near room 11, `move_key` near room 17 — which is itself informative: all 48 verified repairs are `add_door`, confirming the hypothesis. Three candidates touch room 2: `add_door(11,2)`, `add_door(15,2)`, `add_door(17,2)`.

5 · COMPARE t = 25.6 s

Four options side by side: hop distance, topology change, key economy afterwards, the new winning route, rooms opened. `add_door(11,2)` opens 1 new room; `add_door(15,2)` opens 3.

6 · REVISED HYPOTHESIS t = 32.1 s

"The destination should be the boss room rather than the goal. Connecting 2 directly to 11 would bypass the boss; connecting to 17 would enter behind it."

Revision is allowed and logged; the first hypothesis is the one that gets scored.

7 · SUBMIT → GATE → CHECKPOINT t = 35.4 s

`submit(kind="add_door", a="2", b="15", reason=…)` → accepted (it is in the verified set) → `human_checkpoint: approved, automatic: true`.

Scored: intent_hit = false. Layout preserved 0.955. 6 model calls, 19,423 input tokens, 1,925 output, **$0.0062**, 35.4 s.

WHY THIS TRACE IS WORTH A JUDGE'S TIME

The diagnosis is correct and well-argued, every tool does its job, and the submit gate guarantees a valid, completable dungeon. The answer is still wrong, because choosing *which* corridor was dropped needs a signal about where rooms sat relative to each other — the exact gap named in §11 and §14. **The system failed in the place it is designed to fail, and the trace shows you where.**

Dungeon Repair · Kabyik Kayal · micro1 Frontier Engineering Challenge 2026 · 31 August 2026
Code, changelog, reproduction guide and 539 agent trajectories: github.com/Kabyik-Kayal/dungeon-repair